UNIDAD 4. UTILIZACIÓN DE OBJETOS Y DESARROLLO DE CLASES

Índice

1. Uso de objetos (Consumo) 2
   1.1 Características y ciclo de vida 2
   1.2 Instanciación: declaración y creación 4
   1.3 Interacción con el objeto 6
   1.4 Miembros Estáticos (`static`) 8
   1.5 Gestión de Memoria 10
   1.6 Destrucción de objetos y liberación de memoria 12
2. Desarrollo de clases (Definición) 13
   2.1 Estructura de una clase: Sintaxis general y miembros 13
   2.2 Creación de atributos: Declaración, Inicialización y Ámbito 15
   2.3 Definición de Constructores 17
   2.4 Definición de Métodos 20
   2.5 Visibilidad y Encapsulamiento 22
   2.6 Mecánica de parámetros: Paso por Valor y Referencia 26
   2.7 Reutilización de código 28
3. Organización y Documentación 31
   3.1 Paquetes (`packages`): Organización lógica y control de acceso 31
   3.2 Librerías: Importación, creación y uso 33
   3.3 Documentación: Generación de documentación técnica (Javadoc) 35

Resumen 37
   1. Uso de Objetos (El Rol de Consumidor) 37
   2. Desarrollo de Clases (El Rol de Arquitecto) 38
   3. Organización y Documentación (El Rol de Bibliotecario) 39

1. Uso de objetos (Consumo)

1.1 Características y ciclo de vida

Un objeto en Java no es una idea abstracta; es un trozo físico de memoria RAM reservado para guardar datos. Todo objeto que "vive" en tu programa pasa por un ciclo de nacimiento, vida y muerte.

A. Características de un Objeto en Ejecución

Cuando el programa está corriendo, cada objeto posee tres características fundamentales que lo definen:

1. **Estado (Atributos):** Son los datos que guarda en ese preciso instante.
    - *Ejemplo:* Un objeto `Coche` puede tener velocidad = 0 y color = "Rojo". Si acelera, su estado cambia (velocidad = 20), pero sigue siendo el mismo objeto.
2. **Comportamiento (Métodos):** Lo que el objeto es capaz de hacer.
    - *Ejemplo:* `arrancar()`, `frenar()`.
3. **Identidad (Referencia):** Es lo que lo distingue de los demás, incluso si dos objetos tienen los mismos datos. En Java, la identidad está ligada a su dirección de memoria.
    - *Ejemplo:* Puedes tener dos coches rojos idénticos en un parking. Son iguales en **estado**, pero son dos objetos distintos (tienen distinta **identidad** o matrícula).

B. El Ciclo de Vida de un Objeto

Desde que decides que necesitas un objeto hasta que desaparece de la memoria, ocurre una secuencia de pasos estricta.

Podemos dividir el ciclo en 3 fases principales:

Fase 1: Creación (Nacimiento)

Es el momento en que el sistema operativo reserva espacio en la memoria para el objeto. Aquí intervienen tres conceptos que a menudo se confunden, pero son pasos distintos:

1. **Declaración:** Creas la etiqueta (variable), pero está vacía.
    - `Coche miCoche;` (Aquí aún no existe el objeto, solo la referencia).
2. **Instanciación:** Usas la palabra clave `new`. Esto reserva la memoria en el "Heap".
    - `new Coche`
3. **Inicialización:** Se llama al **Constructor** `()` para dar los valores iniciales.
    - `...Coche();`

Sintaxis completa: `Coche miCoche = new Coche();`

Fase 2: Utilización (Vida)

Es el periodo en el que el objeto está activo en memoria y lo usamos para realizar tareas. Accedemos a sus miembros (métodos y atributos) utilizando el operador punto (.).

- `miCoche.color = "Azul";` (Modificamos su estado).
- `miCoche.arrancar();` (Invocamos su comportamiento).

Mientras exista al menos una variable (referencia) apuntando al objeto, este seguirá vivo.

Fase 3: Destrucción (Muerte)

A diferencia de lenguajes antiguos (como C++) donde tú tenías que borrar la memoria manualmente (lo cual causaba muchos errores), Java gestiona la muerte de los objetos automáticamente.

- **Pérdida de Referencia:** Un objeto deja de ser útil cuando ya nadie apunta a él.
    - **Ejemplo:** `miCoche = null;` (Rompes el vínculo).
    - **Ejemplo:** Se termina el método donde creaste el objeto (la variable local muere y suelta el objeto).
- **Garbage Collector (Recolector de Basura):** Es un programa de limpieza de Java que pasa periódicamente. Si ve un objeto sin referencias (abandonado), libera esa memoria para que pueda ser usada por otros.

Resumen Visual: Referencia vs Objeto

Es vital distinguir entre la variable y el objeto.

- **La Variable (`miCoche`):** Es un mando a distancia. Vive en el Stack (memoria rápida).

- **El Objeto (`new Coche()`):** Es el televisor. Vive en el Heap (memoria grande).

Si haces `miCoche = null`, no rompes el televisor, simplemente tiras el mando a la basura. El televisor (objeto) se queda ahí hasta que venga el basurero (Garbage Collector) y se lo lleve.

1.2 Instanciación: declaración y creación

A menudo llamamos "crear un objeto" a todo el proceso, pero técnicamente son tres pasos distintos que ocurren en milisegundos.

Tomemos esta línea común:

Java
```
Scanner lector = new Scanner(System.in);
```

Vamos a diseccionarla en sus tres componentes:

1. Declaración (`Scanner lector`)

Aquí le estás diciendo a Java: *"Resérvame un hueco en la memoria Stack para guardar una referencia (un mando a distancia) de tipo Scanner"*.

- **Estado:** En este punto, la variable `lector` existe, pero no apunta a nada. Su valor es `null` (nulo).
- **Peligro:** Si intentas usarla ahora (`lector.nextLine()`), el programa explotará (`NullPointerException`), porque el mando no está conectado a ninguna tele.

2. Creación / Instanciación (`new`)

La palabra clave **`new`** es el operador más importante de la POO.

- **Acción:** Le pide al sistema operativo un trozo de memoria libre en el **Heap** (el almacén grande).
- **Resultado:** Crea físicamente el objeto.
- **Retorno:** Devuelve la **dirección de memoria** donde ha guardado ese objeto (ej: `0x45F3A`).

3. Inicialización (`Scanner(...)`)

Justo después de reservar la memoria con `new`, Java llama automáticamente a un método especial: el **Constructor**. Su trabajo es preparar el objeto para su primer uso (poner los contadores a 0, limpiar buffers, etc.).

¿Qué es exactamente un Constructor?

El **Constructor** es un método especial que tiene la misión exclusiva de **inicializar** el objeto. Es lo que diferencia un bloque de memoria vacío de un objeto útil y configurado.

Características Clave (Desde el punto de vista del uso):

1. **Mismo nombre que la clase:** Si la clase es `Coche`, el constructor se llama `Coche()`.
2. **Se invoca con `new`:** No puedes llamar a un constructor manualmente como si fuera un método normal (no puedes hacer `miObjeto.Coche()`). Solo se ejecuta una vez al nacer el objeto.
3. **No devuelve nada:** No tiene `void`, ni `int`, ni nada. Devuelve (implícitamente) la instancia creada.

Tipos de Constructores

Cuando usas clases creadas por otros (o librerías de Java), te encontrarás dos situaciones:

A. El Constructor Vacío (o por defecto), No recibe parámetros. Crea un objeto "estándar" o "en blanco".

Java
```
// Crea un String vacío
String texto = new String();
```

B. El Constructor con Parámetros. Esta es la gran ventaja. Te permite crear el objeto y configurarlo en la misma línea. Le pasas los datos iniciales entre los paréntesis.

Java
```
// Crea un String que YA contiene "Hola" desde su nacimiento
String saludo = new String("Hola");

// Imaginemos una clase Rectangulo
// Crea el objeto y ya le asigna ancho=10 y alto=20
Rectangulo r = new Rectangulo(10, 20);
```

Ejemplo práctico paso a paso

Imagina que tenemos una clase `Videojuego`. Veamos cómo se instancia en memoria.

Java
```
// 1. Declaración
Videojuego juegoFavorito;
// En Stack: [juegoFavorito: null]

// 2. Creación e Inicialización
juegoFavorito = new Videojuego("Minecraft");
// a. 'new' reserva memoria en Heap.
// b. 'Videojuego("Minecraft")' guarda el nombre dentro del objeto.
// c. El operador '=' conecta la variable del Stack con la
dirección del Heap.
```

La importancia de `null`

Si declaras una variable de objeto, pero no haces el `new`, la variable apunta a la nada.

Java
```
Coche miFerrari; // Declarado, pero no instanciado
// miFerrari.arrancar(); // ¡ERROR FATAL! NullPointerException
```

Analogía del Mando a Distancia:

- **Declaración:** Compras un mando universal (`miFerrari`). Aún no sirve para nada.
- **Instanciación (`new`):** Compras la televisión y la pones en el salón.
- **Asignación (=):** Sincronizas el mando con la tele. Ahora, si aprietas un botón en el mando, la tele reacciona.

1.3 Interacción con el objeto

Una vez que tenemos una referencia apuntando a un objeto (ej: `miCoche`), utilizamos el **Operador Punto (.)** para acceder a sus miembros internos. Piensa en el punto como la llave que abre la puerta del objeto.

A través de este operador, realizamos dos tipos de interacciones: manipular sus datos (**Propiedades**) o activar sus comportamientos (**Métodos**).

A. Acceso y modificación de Propiedades (Atributos)

Las propiedades definen el **estado** actual del objeto. Interactuar con ellas implica leerlas o cambiarlas.

1. Modificación (Escritura): Asignamos un valor a un atributo del objeto. Esto cambia el estado interno de esa instancia específica.

- **Sintaxis:** `objeto.atributo = nuevoValor;`

2. Acceso (Lectura): Recuperamos el valor que está guardado actualmente para usarlo en cálculos, impresiones, etc.

- **Sintaxis:** `tipo variable = objeto.atributo;`

Ejemplo: Imaginemos una clase `Producto` con atributos públicos.

Java
```
Producto p = new Producto();

// MODIFICACIÓN: Cambiamos su estado
p.nombre = "Portátil";
p.precio = 500.0;

// ACCESO: Leemos su estado para usarlo
double precioConIVA = p.precio * 1.21;
System.out.println("El producto es: " + p.nombre);
```

Importante: En el desarrollo profesional, es raro acceder directamente a los atributos (como p.precio). Lo normal es que los atributos sean private (privados) para protegerlos. En ese caso, la "interacción con propiedades" se realiza indirectamente a través de métodos especiales llamados Getters (para leer) y Setters (para modificar), como veremos a continuación.

B. Invocación de Métodos

Invocar un método es enviar un mensaje al objeto para que realice una acción. Aquí entran en juego dos conceptos clave: qué le enviamos nosotros (Parámetros) y qué nos devuelve él (Retorno).

1. Paso de Parámetros (Input). Muchos métodos necesitan datos externos para funcionar. Estos datos se colocan entre paréntesis.

- Debes respetar estrictamente el **orden**, el **tipo** y la **cantidad** de los argumentos definidos en la clase.

Java
```
// Método: .sumar(int a, int b)
calculadora.sumar(5, 10); // ✅ Correcto
// calculadora.sumar("Hola"); // ❌ Error: Espera números, no texto
// calculadora.sumar(5);      // ❌ Error: Espera 2 datos, no 1
```

2. Captura de Valores de Retorno (Output). Cuando el método termina, puede devolver un resultado. Tenemos dos casos:

- **Métodos `void` (Sin retorno):** Realizan una acción pero no devuelven ningún dato. No puedes asignar su resultado a una variable.

Java
```
// Solo se ejecuta la acción
usuario.enviarEmail();
```

- **Métodos con Retorno (Tipados):** Devuelven un dato (`int`, `String`, `boolean`, etc.). Debes "capturar" ese dato en una variable si quieres usarlo, o usarlo directamente en una expresión.

Java
```
// El método .generarPassword() devuelve un String
String nuevaPass = usuario.generarPassword(); // Capturamos el dato

// Si no lo capturas:
usuario.generarPassword(); // El cálculo se hace, pero el resultado
se pierde inmediatamente.
```

C. Ejemplo Integrado: Propiedades y Métodos

Veamos un ejemplo completo con una clase `CuentaBancaria` que mezcla ambos conceptos.

Java
```
// 1. Instanciación
CuentaBancaria miCuenta = new CuentaBancaria();

// --- INTERACCIÓN CON PROPIEDADES (Directa) ---
// (Suponiendo que 'titular' es público)
miCuenta.titular = "Juan Pérez"; // Modificación

// --- INTERACCIÓN CON MÉTODOS ---
// Invocación con parámetro (Input)
miCuenta.ingresar(500);
```

```java
// Invocación que devuelve valor (Output)
// Supongamos que .retirar() devuelve true si hubo saldo suficiente
boolean exito = miCuenta.retirar(100);

if (exito) {
    System.out.println("Retirada realizada");
}

// Invocación para leer una propiedad privada (Getter)
double saldoActual = miCuenta.getSaldo(); // Captura de retorno
```

1.4 Miembros Estáticos (`static`)

La palabra clave **`static`** cambia las reglas del juego.

- **Sin `static` (Miembro de Instancia):** Pertenece a **cada objeto** individual. Cada objeto tiene su propia copia.
- **Con `static` (Miembro de Clase):** Pertenece a la **Clase** en sí misma. Solo existe **una única copia** compartida por todos los objetos.

A. Atributos Estáticos (Variables compartidas)

Imagina que son variables globales para todos los objetos de ese tipo. Si un objeto cambia el valor de una variable estática, **cambia para todos los demás objetos**.

Analogía:

- **Atributo de Instancia (`nombre`):** Es como tu cepillo de dientes. Cada persona tiene el suyo propio.
- **Atributo Estático (`contador`):** Es como el interruptor de la luz del baño. Hay uno solo para todos. Si tú lo apagas, se apaga para todos.

Ejemplo Práctico: Contar cuántos marcianos hemos creado.
Java

```java
class Marciano {
    // Variable de Instancia: Cada marciano tiene su propio nombre.
    String nombre;

    // Variable Estática: COMPARTIDA. Cuenta el total de marcianos.
    static int cantidadTotal = 0;

    public Marciano(String nombre) {
        this.nombre = nombre;
        // Cada vez que nace uno, sumamos 1 al contador compartido
        cantidadTotal++;
    }
}

// USO:
Marciano m1 = new Marciano("Zorg");
Marciano m2 = new Marciano("Gorp");

// ¿Cuántos hay?
```

```
// Accedemos a través de la CLASE, no del objeto.
System.out.println(Marciano.cantidadTotal); // Imprime: 2
```

B. Métodos Estáticos (Herramientas universales)

Son métodos que realizan una tarea sin necesitar datos de un objeto específico. Son "funciones puras" o utilidades. Para usarlos, no hace falta hacer new. Se llaman usando el nombre de la clase.

El ejemplo clásico es la clase `Math` de Java. No creas un objeto "Matemáticas", simplemente usas sus herramientas.

Java
```
// Forma INCORRECTA (y a veces imposible):
// Math m = new Math();
// m.sqrt(25);

// Forma CORRECTA (Usando la Clase):
double raiz = Math.sqrt(25); // Llamada estática
```

La Regla de Oro de los Métodos Estáticos: Un método estático NO puede acceder a atributos de instancia (no estáticos).

- *¿Por qué?* Porque el método estático no sabe "quién" lo está llamando. Si haces `Clase.metodo()`, no hay ningún objeto `this` asociado. No puede saber el `nombre` de nadie porque no está vinculado a ninguna instancia concreta.

Resumen: Diferencias Clave

Característica	De Instancia (Sin static)	De Clase (Static)
Pertenencia	Pertenece al objeto individual (`new`).	Pertenece a la definición de la Clase.
Copias en memoria	Una copia por cada objeto creado (N copias).	**Una sola copia** para toda la aplicación.
Acceso	`objeto.metodo()`	`Clase.metodo()`
Acceso a datos	Puede acceder a todo (static y no static).	Solo puede acceder a otros miembros static.
Ejemplo	`miCoche.getColor()`	`Math.random()`

1.5 Gestión de Memoria

Java divide la memoria RAM asignada a tu programa en dos grandes áreas principales: el **Stack** (Pila) y el **Heap** (Montículo). Cada una tiene un comportamiento y propósito totalmente distinto.

1. El Stack (La Pila)

Es la zona de memoria **rápida, ordenada y temporal**.

- **¿Qué guarda?:** Las llamadas a los métodos y las **variables locales** (las que creas dentro de los métodos).
- **Comportamiento:** Funciona como una pila de platos (LIFO - Last In, First Out). Cuando entras en un método, se añade un bloque arriba; cuando sales, ese bloque se destruye inmediatamente.
- **Lo vital:** Aquí viven los **Tipos Primitivos** (`int`, `double`, `boolean`) y las **Referencias** (los mandos a distancia) de los objetos.

2. El Heap (El Montículo)

Es la zona de memoria **grande, desordenada y persistente**.

- **¿Qué guarda?:** Los **OBJETOS** reales. Todo lo que creas con la palabra `new` vive aquí.
- **Comportamiento:** Es como un almacén gigante. Los objetos se quedan aquí hasta que el recolector de basura (Garbage Collector) los borra.
- **Acceso:** Es más lento que el Stack. Para llegar a un dato aquí, Java tiene que seguir la "flecha" (dirección de memoria) desde el Stack.

La Regla de Oro del Almacenamiento

Para entender dónde está cada cosa, mira el tipo de dato:

1. **Si es Primitivo (`int x = 10;`):**

   - La variable `x` está en el **Stack**.
   - El valor `10` está **dentro** de la variable, en el **Stack**.

2. **Si es Objeto (`Coche c = new Coche();`):**

   - La variable `c` (la referencia) está en el **Stack**.
   - El objeto real (`{color: rojo, velocidad: 0}`) está en el **Heap**.
   - La variable `c` solo guarda la **dirección** (ej: `0x5F3`) para encontrarlo.

¿Qué ocurre realmente al asignar variables?

Aquí es donde nacen el 90% de los errores de principiante. La asignación (=) funciona diferente según el tipo.

A. Asignación de Primitivos (Copia de Valor). Cuando igualas dos variables primitivas, se hace una fotocopia del valor. Son independientes.

Java
```
int a = 5;
int b = a;  // Hacemos una COPIA del 5 en 'b'.

b = 10;     // Cambiamos 'b'.

// RESULTADO:
// a sigue valiendo 5.
// b vale 10.
// (Son dos hojas de papel distintas).
```

B. Asignación de Objetos (Copia de Referencia / Aliasing). Cuando igualas dos variables de objeto, **NO se copia el objeto**. Se copia la dirección de memoria (el mando a distancia). Ambas variables apuntan al **MISMO** objeto en el Heap. Esto se llama **Aliasing**.

Java
```
Coche c1 = new Coche();
c1.color = "Rojo";

// ¡ATENCIÓN AQUÍ!
Coche c2 = c1;
// NO hemos creado un coche nuevo.
// Hemos copiado el mando. Ahora c1 y c2 apuntan al MISMO coche.

c2.color = "Azul"; // Pintamos el coche usando el mando 2.

// PREGUNTA: ¿De qué color es c1?
System.out.println(c1.color);
// SALIDA: "Azul"
// (¡El coche rojo ha desaparecido porque c1 y c2 son el mismo
coche!)
```

Comparación Visual

Acción	Primitivos (int, char...)	Objetos (String, Coche...)
Memoria	Todo en el Stack.	Referencia en Stack, Objeto en Heap.
Asignación (=)	**Clona** el valor.	**Comparte** la referencia.
Efecto	Variables independientes.	Variables conectadas (Alias).
Comparación (==)	Compara valores (¿5 es igual a 5?).	Compara direcciones (¿Apuntan al mismo sitio?).

Más info: Aunque `String` es un objeto, Java tiene trucos para que parezca primitivo a veces, pero recuerda: **se comporta como un objeto**. Si usas `==` con Strings, a veces falla porque compara direcciones de memoria, no el texto. (Para comparar texto siempre usa `.equals()`).

1.6 Destrucción de objetos y liberación de memoria

En lenguajes antiguos como C o C++, el programador era responsable de **crear** la memoria y también de **borrarla** manualmente. Si se te olvidaba borrarla, el programa iba consumiendo RAM hasta colapsar el ordenador (esto se llama *Memory Leak* o Fuga de Memoria).

Java revolucionó esto introduciendo la **Gestión Automática de Memoria**. Tú solo te preocupas de crear objetos. Java se preocupa de destruirlos.

A. El concepto de Referencias Perdidas

Para entender cuándo muere un objeto, debes recordar la regla de oro del Heap:

> **"Un objeto tiene derecho a vivir en el Heap solo mientras alguien (una variable en el Stack) lo esté apuntando."**

En el momento en que un objeto se queda sin referencias (nadie tiene su "mando a distancia"), se convierte en **basura** (Garbage). Es inaccesible. Ocupa espacio, pero no se puede usar.

¿Cómo se pierden las referencias?

1. **Nulificar la referencia:** Asignas null a la variable. Rompes el vínculo explícitamente.

Java
```
Coche c1 = new Coche(); // El objeto Coche vive
c1 = null;              // El objeto Coche ha sido abandonado.
Nadie lo apunta.
```

2. **Reasignación:** Haces que la variable apunte a otro sitio. El objeto antiguo queda suelto.

Java
```
String s = new String("Hola");
s = new String("Adios"); // "Hola" se queda sin referencia. Muere.
```

3. **Fin del alcance (Scope):** Cuando un método termina, todas sus variables locales (del Stack) desaparecen. Los objetos que esas variables apuntaban en el Heap se quedan huérfanos.

B. El Garbage Collector (Recolector de Basura)

El **Garbage Collector (GC)** es un proceso (un hilo de ejecución) de la Máquina Virtual Java (JVM) que corre en segundo plano. Es como el servicio de limpieza de la ciudad.

¿Cómo funciona?

1. Periódicamente, el GC escanea la memoria Heap.
2. Identifica los objetos que tienen **cero referencias** activas.
3. Los elimina físicamente, liberando esa memoria RAM para que pueda ser usada por nuevos objetos.

Características importantes:

- **Es No-Determinista:** No sabes cuándo va a pasar. Puede pasar dentro de 1 milisegundo o dentro de 1 minuto. La JVM decide cuándo es el mejor momento (normalmente cuando la memoria empieza a llenarse).
- **No puedes forzarlo:** Existe un comando `System.gc();` que *sugiere* a la JVM que pase el recolector, pero la JVM puede ignorar tu sugerencia si considera que no es eficiente hacerlo en ese momento.

Más info: En Java, no existe un comando `delete` o `free`. Para destruir un objeto, simplemente deja de usarlo (asegúrate de que ninguna variable lo apunte) y el sistema se encargará del resto.

2. Desarrollo de clases (Definición)

2.1 Estructura de una clase: Sintaxis general y miembros

Una **Clase** es la unidad fundamental de la programación en Java. Técnicamente, es una plantilla o molde que define la forma y el comportamiento que tendrán los objetos creados a partir de ella.

A. Sintaxis General

En Java, la definición de una clase sigue una estructura estricta. Todo el código debe estar encerrado entre unas llaves principales `{ }`.

Reglas de estilo:

- Se usa la palabra reservada **`class`**.
- El nombre debe seguir **PascalCase** (PrimeraLetraMayuscula).
- El nombre del archivo debe coincidir **exactamente** con el nombre de la clase (ej: `Coche.java`).

Java
```
// Estructura Esqueleto
public class NombreDeLaClase {
```

```
    // 1. Zona de ATRIBUTOS (Variables)
    // ...

    // 2. Zona de CONSTRUCTORES (Inicialización)
    // ...

    // 3. Zona de MÉTODOS (Funciones)
    // ...

}
```

B. Los Miembros de una Clase

Dentro de las llaves de la clase, definimos sus componentes internos, conocidos como **miembros**. Los dos principales son los Atributos y los Métodos.

1. Atributos (El Estado / "Lo que ES")

También llamados "campos", "propiedades" o "variables de instancia". Son variables que se declaran **dentro de la clase, pero fuera de cualquier método.**

- **Función:** Almacenan los datos individuales de cada objeto. Si creas 10 objetos, habrá 10 copias de estas variables (una para cada uno).
- **Naturaleza:** Suelen ser sustantivos o adjetivos (nombre, edad, color, saldo).

2. Métodos (El Comportamiento / "Lo que HACE")

Son funciones definidas dentro de la clase.

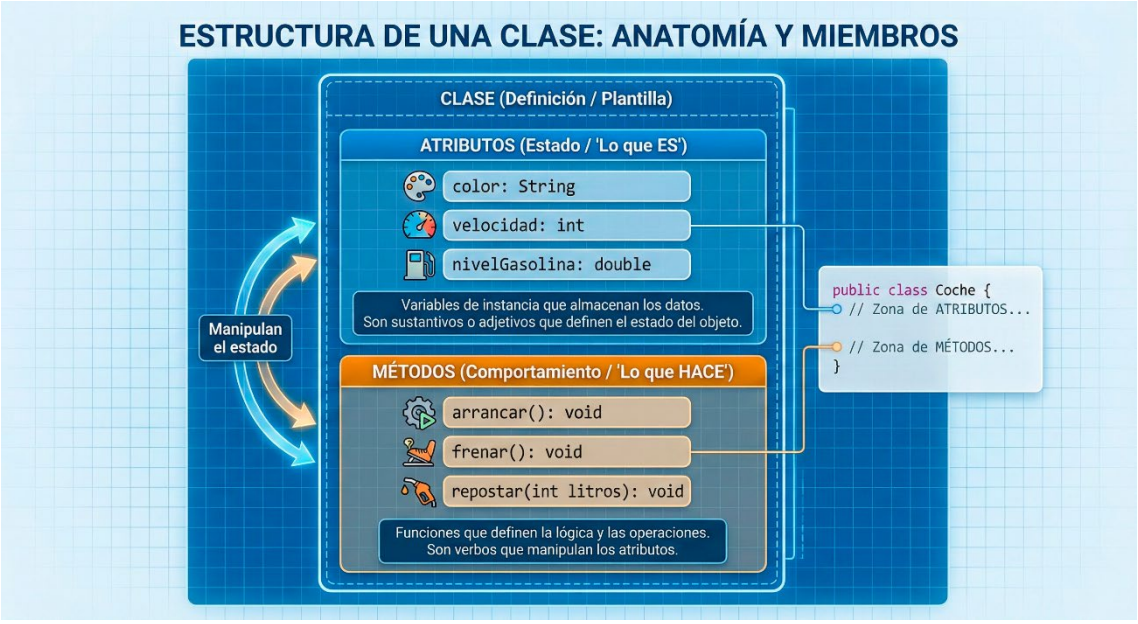
- **Función:** Definen la lógica y las operaciones que el objeto puede realizar. Manipulan los atributos del objeto para cambiar su estado.
- **Naturaleza:** Suelen ser verbos (correr, calcular, guardar, imprimir).

C. Ejemplo Completo: Anatomía de una Clase

Vamos a diseñar una clase que represente un `Libro` en una biblioteca. Fíjate en cómo se separan los datos de las acciones.

Java
```
public class Libro {

    // --- MIEMBROS TIPO: ATRIBUTOS (Estado) ---
    // Variables que definen qué es un libro.
    String titulo;
    String autor;
    int numeroPaginas;
    boolean prestado; // true si está prestado, false si está
disponible

    // --- MIEMBROS TIPO: MÉTODOS (Comportamiento) ---
    // Acciones que se pueden hacer con el libro.
```

```
    // Método para marcar el libro como prestado
    void prestar() {
        prestado = true;
        System.out.println("El libro " + titulo + " ha sido
prestado.");
    }

    // Método para devolver el libro
    void devolver() {
        prestado = false;
        System.out.println("El libro " + titulo + " ha sido
devuelto.");
    }

    // Método que nos da información (Reporte)
    void mostrarInformacion() {
        System.out.println("Libro: " + titulo + " de " + autor);
        System.out.println("¿Está prestado?: " + prestado);
    }
}
```



Más info: Aunque Java permite mezclar atributos y métodos, el estándar profesional (convención) es declarar **primero todos los atributos** en la parte superior del archivo, y **después los métodos**. Esto facilita la lectura: primero ves qué datos maneja la clase y luego cómo los usa.

2.2 Creación de atributos: Declaración, Inicialización y Ámbito

A. Declaración e Inicialización por defecto

Los atributos se declaran dentro de la clase, pero **fuera** de cualquier método.

La Regla de Oro de la Inicialización: A diferencia de las variables locales (las que creas dentro de un método), **los atributos NO necesitan ser inicializados obligatoriamente.**

Si tú no les das un valor, Java es "amable" y les asigna un **Valor por Defecto** (Default Value) al crear el objeto, para asegurar que la memoria esté limpia.

Tipo de Dato	Valor por Defecto (Si tú no pones nada)
Números (`byte`, `short`, `int`, `long`)	`0`
Decimales (`float`, `double`)	`0.0`
Booleano (`boolean`)	`false`
Carácter (`char`)	`\u0000` (Carácter nulo / vacío)
Objetos / Referencias (`String`, `Coche`...)	`null` (Peligroso)

Ejemplo demostrativo:
Java
```java
public class Jugador {
    // Declaramos atributos SIN darles valor inicial
    String nombre;
    int puntos;
    boolean activo;

    void mostrarEstado() {
        // En una variable local, esto daría ERROR de compilación.
        // En un atributo, Java pone los valores por defecto.
        System.out.println("Nombre: " + nombre);  // Imprime: null
        System.out.println("Puntos: " + puntos);  // Imprime: 0
        System.out.println("Activo: " + activo);  // Imprime: false
    }
}
```

Nota: Aunque Java les dé un valor, es buena práctica inicializarlos tú mismo si quieres que empiecen con un valor lógico (ej: `int vidas = 3;`).

B. Ámbito (Scope): Atributos vs Variables Locales

Este es un concepto fundamental para evitar errores lógicos. El **ámbito** define **dónde** se puede usar una variable y **cuánto tiempo** vive.

En una clase conviven dos tipos de variables "enemigas":

1. **Variables de Instancia (Atributos):**

   - **Dónde viven:** En el **Heap** (dentro del objeto).
   - **Alcance:** Son visibles en **TODA la clase**. Cualquier método de la clase puede usarlas.
   - **Vida:** Nacen con el `new` y mueren cuando el objeto es recolectado por la basura.

2. **Variables Locales:**

- **Dónde viven:** En el **Stack** (memoria temporal).
- **Alcance:** Son visibles **SOLO dentro del método** (o bloque) donde se crearon. Fuera de esas llaves `{ }`, no existen.
- **Vida:** Mueren instantáneamente al terminar el método.
- **Inicialización: OBLIGATORIA**. Java no les da valor por defecto.

Tabla Comparativa:

Característica	Atributo (Campo)	Variable Local
Declaración	Dentro de la clase, fuera de métodos.	Dentro de un método.
Visibilidad	Toda la clase.	Solo su propio método.
Valor Inicial	Automático (0, null, false).	**Ninguno** (Error si no la inicias).
Almacenamiento	Heap.	Stack.

C. El conflicto de nombres (Shadowing / Ocultación)

¿Qué pasa si un atributo y una variable local se llaman igual? Java da prioridad a la variable local (la más cercana). A esto se le llama "Ocultación de variable" (Variable *Shadowing*).

Para solucionar esto y referirnos al atributo, usamos la palabra clave **`this`**.

- **`this.variable`** = El Atributo del objeto.
- **`variable`** = La variable local.

Ejemplo Clásico:
Java
```
public class Persona {
    String nombre; // ATRIBUTO

    // El parámetro 'nombre' es una VARIABLE LOCAL
    public void setNombre(String nombre) {

        // nombre = nombre; // ❌ MAL: Estás asignando la local a
sí misma. El atributo no cambia.

        this.nombre = nombre; // ✅ BIEN: "Guarda la local 'nombre'
en MI atributo 'nombre'"
    }
}
```

2.3 Definición de Constructores

Un constructor es un bloque de código especial que se ejecuta **automáticamente** y **una sola vez** en el momento en que se invoca la palabra clave `new`.

Reglas de Sintaxis (Obligatorias):

1. **Mismo nombre:** Debe llamarse **exactamente igual** que la clase.
2. **Sin retorno:** No puede tener ningún tipo de retorno, ni siquiera `void`. Si le pones `void`, Java pensará que es un método normal y no funcionará como constructor.

A. El Constructor por Defecto (El "Invisible")

Este es un concepto que suele causar confusión en los exámenes. Si tú escribes una clase y **NO escribes ningún constructor**, Java crea uno por ti de forma invisible durante la compilación.

- **¿Qué hace?** Nada. Es un constructor vacío `{ }`.
- **¿Para qué sirve?** Simplemente para permitirte hacer `new Clase()` sin errores.
- **Resultado:** Los atributos se quedan con sus valores por defecto (0, null, false).

Java
```java
public class Coche {
    String marca;

    // Java inserta esto invisiblemente si tú no escribes nada:
    // public Coche() {
    // }
}

// En el main:
Coche c = new Coche(); // Funciona gracias al constructor por defecto.
```

B. Constructores Personalizados (Tomando el control)

Lo habitual es que queramos obligar a quien use nuestra clase a dar valores iniciales. Para eso, definimos nuestro propio constructor con parámetros.

Usamos la palabra clave **`this`** para diferenciar entre el atributo del objeto y el parámetro que nos pasan.

Java
```
public class Coche {
    String marca;
    String modelo;
    int velocidad;

    // Constructor Personalizado
    public Coche(String marca, String modelo) {
        this.marca = marca;     // Asigna el parámetro 'marca' al
atributo 'marca'
        this.modelo = modelo;   // Asigna el parámetro 'modelo' al
atributo 'modelo'
        this.velocidad = 0;     // Podemos poner valores fijos
lógicos
    }
}
```

La Regla de la Desaparición del Default:

> **¡MUY IMPORTANTE!** En el momento en que escribes cualquier constructor (aunque sea uno con parámetros), Java deja de insertar el constructor por defecto.
>
> Si escribes `public Coche(String marca) { ... }`, la línea `new Coche()` (sin parámetros) dejará de funcionar y dará error de compilación, a menos que tú escribas manualmente también el constructor vacío.

C. Sobrecarga de Constructores (Varias formas de nacer)

Java permite tener **más de un constructor** en la misma clase, siempre y cuando tengan **diferentes parámetros** (en cantidad o tipo). Esto da flexibilidad al crear objetos.

Ejemplo completo con una clase `Usuario`:
Java
```
public class Usuario {
    String nombre;
    String rol;
    boolean activo;

    // Opción 1: Constructor Completo (El más estricto)
    public Usuario(String nombre, String rol) {
        this.nombre = nombre;
        this.rol = rol;
        this.activo = true; // Por defecto nace activo
    }

    // Opción 2: Constructor Mínimo (El "vago")
    // Si no nos dicen el rol, asumimos que es "Invitado".
    public Usuario(String nombre) {
        this.nombre = nombre;
```

```
        this.rol = "Invitado";
        this.activo = true;
    }

    // Opción 3: Constructor Vacío (Recuperamos el default
manualmente)
    // Útil si queremos crear el objeto primero y rellenarlo
después con setters.
    public Usuario() {
        this.nombre = "Sin Nombre";
        this.rol = "Sin Rol";
        this.activo = false;
    }
}
```

¿Cómo se usa esto en el Main? El programador elige qué constructor usar según los datos que tenga:

Java
```
// Usa la Opción 1
Usuario u1 = new Usuario("Ana", "Admin");

// Usa la Opción 2
Usuario u2 = new Usuario("Pepe");

// Usa la Opción 3
Usuario u3 = new Usuario();
```

Al diseñar una clase, pregúntate: **¿Qué datos son IMPRESCINDIBLES para que este objeto exista?**

- Si un `Coche` no tiene sentido sin `marca`, crea un constructor que pida la marca obligatoriamente.
- Si un `Email` no tiene sentido sin `destinatario`, oblígalo en el constructor.

Esto evita que existan objetos "inválidos" o vacíos circulando por tu programa.

2.4 Definición de Métodos

Un método es un bloque de código reutilizable que realiza una tarea específica. Definirlo correctamente es crucial para que tu código sea limpio y modular.

A. Declaración y Estructura

La definición de un método sigue una "anatomía" muy concreta.

Sintaxis General:
Java
```
[Modificador] [TipoRetorno] nombreMetodo([ListaDeArgumentos]) {
    // CUERPO DEL MÉTODO
    // Instrucciones...
    [return valor;]
```

```
}
```

1. **Modificador:** Define la visibilidad (ej: `public`, `private`). Lo veremos en el siguiente punto.
2. **Tipo de Retorno:** Indica qué tipo de dato nos va a devolver la máquina.
   - Si devuelve algo: `int`, `String`, `double`, `Coche`...
   - Si NO devuelve nada: **`void`**.
3. **Nombre:** Verbo en minúscula (camelCase). Ej: `calcularTotal`, `abrirPuerta`.
4. **Lista de Argumentos:** Los ingredientes que necesita (Tipo y Nombre).

B. Argumentos y Valores de Retorno

Esta es la tubería de entrada y salida del método.

1. Lista de Argumentos (Input). Son variables locales que reciben valores desde fuera. Puedes tener cero, uno o muchos, separados por comas.

- `()`: Sin parámetros.
- `(int a)`: Un parámetro.
- `(String n, int e, boolean x)`: Múltiples parámetros.

2. El Valor de Retorno (Output). Es la respuesta final del método.

- Si declaras un tipo (ej: `int`), es **OBLIGATORIO** usar la palabra clave **`return`** seguida de un valor de ese tipo antes de que cierre la llave.
- Si usas `void`, el `return` es opcional (se usa solo para salir antes de tiempo) y no lleva valor.

Ejemplo Práctico:
Java
```java
public class Calculadora {

    // 1. Método VOID (Hace una acción, no devuelve dato)
    // Argumento: String mensaje
    public void imprimirAlerta(String mensaje) {
        System.out.println("ALERTA: " + mensaje);
        // No hay return
    }

    // 2. Método con RETORNO (Calcula y devuelve)
    // Argumentos: dos doubles
    // Retorno: un double
    public double sumar(double a, double b) {
        double resultado = a + b;
        return resultado; // OBLIGATORIO: Devuelve el dato y sale
del método
    }

    // 3. Método BOOLEANO (Pregunta algo)
    public boolean esMayorDeEdad(int edad) {
        if (edad >= 18) {
            return true;
        } else {
            return false;
```

```
        }
    }
}
```

C. Sobrecarga de Métodos (Method Overloading)

Java tiene una característica muy potente: permite tener **varios métodos con el mismo nombre** dentro de la misma clase. ¿Cómo sabe Java a cuál llamar? Por la Firma del Método (Signature). La firma se compone de: Nombre del método + Lista de Tipos de Parámetros. Para sobrecargar un método, debes cambiar sus parámetros:

1. Cambiar el **número** de parámetros.
2. Cambiar el **tipo** de parámetros.
3. Cambiar el **orden** de los tipos.

   Lo que NO funciona: Cambiar solo el tipo de retorno o el nombre de las variables no es sobrecarga. Java se confundirá.

Ejemplo: El método `sumar` inteligente. Queremos que nuestra calculadora sepa sumar enteros, decimales o tres números a la vez.

Java
```
public class CalculadoraPro {

    // Versión 1: Suma dos enteros
    public int sumar(int a, int b) {
        return a + b;
    }

    // Versión 2: Suma dos decimales (Cambia el TIPO)
    public double sumar(double a, double b) {
        return a + b;
    }

    // Versión 3: Suma tres enteros (Cambia la CANTIDAD)
    public int sumar(int a, int b, int c) {
        return a + b + c;
    }
}
```

Uso desde el main: Java elige automáticamente la versión correcta según qué le envíes.

Java
```
CalculadoraPro c = new CalculadoraPro();

c.sumar(5, 10);        // Llama a la Versión 1 (int, int)
c.sumar(5.5, 10.2);    // Llama a la Versión 2 (double, double)
c.sumar(1, 2, 3);      // Llama a la Versión 3 (int, int, int)
```

2.5 Visibilidad y Encapsulamiento

Imagina que tu objeto es una casa.

- **Public:** Es la puerta de entrada y el timbre. Cualquiera desde la calle puede usarlos.
- **Private:** Es tu dormitorio o tu caja fuerte. Solo tú (la clase) tienes acceso.
- **Default:** Son las zonas comunes del edificio. Solo los vecinos (el mismo paquete) pueden entrar.

En Java, controlamos quién puede ver y usar cada parte de nuestro código mediante los **Modificadores de Acceso**.

A. Los 4 Niveles de Acceso

Java tiene 4 niveles de visibilidad, ordenados del más restrictivo (cerrado) al más permisivo (abierto).

1. `private` **(Privado) - El más seguro**
   - **Visibilidad:** Visible **SOLO dentro de la misma clase**.
   - **Uso:** Se usa para todos los **atributos** (datos sensibles) y métodos internos que no quieres que nadie más toque.
   - *Nadie fuera de las llaves* `{ }` *de la clase sabe que esto existe.*
2. **Default (Sin palabra clave) - "De paquete"**
   - **Visibilidad:** Visible por cualquier clase que esté **dentro del mismo paquete** (`package`).
   - **Uso:** Se aplica automáticamente si no escribes nada (ni public, ni private).
   - *Es como dejar la puerta abierta a tus vecinos de carpeta.*
3. `protected` **(Protegido) - Herencia**
   - **Visibilidad:** Visible en el mismo paquete **Y TAMBIÉN** por las subclases (hijas) aunque estén en otros paquetes.
   - **Uso:** Lo veremos a fondo en el tema de Herencia. Sirve para dejar "herencia" a las clases hijas.
4. `public` **(Público) - El más abierto**
   - **Visibilidad:** Visible por **CUALQUIER clase** de la aplicación, esté donde esté.
   - **Uso:** Se usa para la **clase** principal y para los **métodos** que quieres que otros usen (tu API o interfaz).

B. Tabla Resumen de Visibilidad

Modificador	Misma Clase	Mismo Paquete	Subclase (Hija)	Mundo Exterior
`private`	✅	❌	❌	❌

Modificador	Misma Clase	Mismo Paquete	Subclase (Hija)	Mundo Exterior
(default)	✅	✅	❌	❌
`protected`	✅	✅	✅	❌
`public`	✅	✅	✅	✅

C. Aplicación Práctica: El Patrón de Encapsulamiento

En el desarrollo profesional, rara vez dejamos los atributos al aire. Seguimos una regla casi sagrada:

> **Regla de Oro:** Haz los atributos private y los métodos de acceso public.

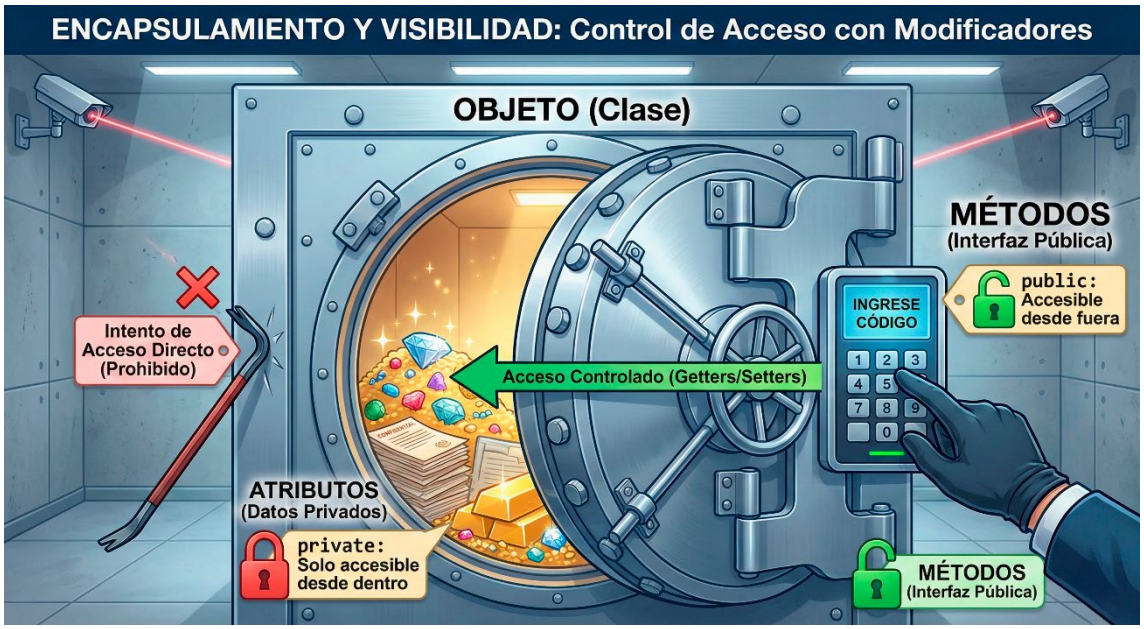
Esto nos permite controlar **cómo** se accede a los datos.

Ejemplo de Mala Práctica (Sin encapsular):
Java
```
public class Cuenta {
    public double saldo; // ❌ PELIGRO: Cualquiera puede poner
saldo negativo
}
// Uso externo:
c.saldo = -5000; // El objeto queda corrupto.
```

Ejemplo de Buena Práctica (Encapsulamiento): Ocultamos el dato real y obligamos a pasar por un "portero" (el método Setter) que valida la información.

Java
```
public class Cuenta {
    // 1. Atributo PRIVADO (Nadie lo toca directamente)
    private double saldo;

    // 2. Método PÚBLICO para leer (Getter)
    public double getSaldo() {
        return this.saldo;
    }

    // 3. Método PÚBLICO para modificar (Setter) con VALIDACIÓN
    public void setSaldo(double cantidad) {
        if (cantidad >= 0) {
            this.saldo = cantidad;
        } else {
            System.out.println("Error: No se permite saldo
negativo.");
        }
    }
}

// Uso externo:
```

```
// c.saldo = -5000; // ❌ ERROR DE COMPILACIÓN (No visible)
c.setSaldo(-5000);  // ✅ El método lo frena e imprime el error.
```



D. Aplicación a Clases y Métodos

No solo los atributos tienen visibilidad, también las piezas grandes:

1. **En Clases (`public class Coche`)**

   - Normalmente las clases son **`public`** para poder usarlas en todo el proyecto.
   - Si quitas el `public` (dejándola *default*), esa clase solo servirá para uso interno dentro de su paquete.

2. **En Métodos Auxiliares**

   - A veces tienes un método complejo que se divide en pasos pequeños. Esos pasos pequeños no le interesan a nadie fuera de la clase.
   - **Estrategia:** Haz el método principal `public` y los pasos pequeños `private`.

Java
```java
public class Motor {

    // Método que el usuario ve (La interfaz)
    public void arrancar() {
        inyectarCombustible(); // Llamada interna
        generarChispa();       // Llamada interna
        System.out.println("Motor arrancado");
    }

    // Métodos internos (Detalles técnicos ocultos)
    private void inyectarCombustible() { ... }
    private void generarChispa() { ... }
}
```

2.6 Mecánica de parámetros: Paso por Valor y Referencia

En Java, existe una regla técnica suprema: **Java SIEMPRE pasa los parámetros "por valor"** (**por copia**). Sin embargo, el efecto es diferente si copias un número o si copias una llave.

A. Paso por Valor (Tipos Primitivos)

Cuando pasas un tipo primitivo (`int, double, boolean, char`) a un método, Java hace una **FOTOCOPIA** exacta del valor. El método trabaja con la fotocopia. Si el método rompe, pinta o cambia la fotocopia, el original (que tienes tú fuera) no sufre ningún cambio.

- **Analogía:** Tienes un papel con el número "10". Haces una fotocopia y se la das a tu amigo. Tu amigo tacha el "10" y escribe "20". Tú miras tu papel original: sigue poniendo "10".

Código de demostración:
Java
```
public class PruebaPrimitivos {

    public void intentarCambiar(int numero) {
        numero = 999; // Cambiamos la COPIA (variable local del
método)
        System.out.println("Dentro del método: " + numero); //
Imprime 999
    }

    public static void main(String[] args) {
        PruebaPrimitivos p = new PruebaPrimitivos();

        int x = 10; // VARIABLE ORIGINAL

        p.intentarCambiar(x); // Pasamos una fotocopia de 'x'

        System.out.println("Fuera del método: " + x); // ¡Sigue
siendo 10!
    }
}
```

Conclusión: Los primitivos son seguros. Un método nunca puede alterar tu variable primitiva original.

B. Paso por Referencia (Objetos)

Aquí es donde ocurre la magia (y el peligro). Cuando pasas un objeto (`String, Coche, Array`) a un método, Java también hace una copia, pero... **copia la REFERENCIA** (**la dirección de memoria**).

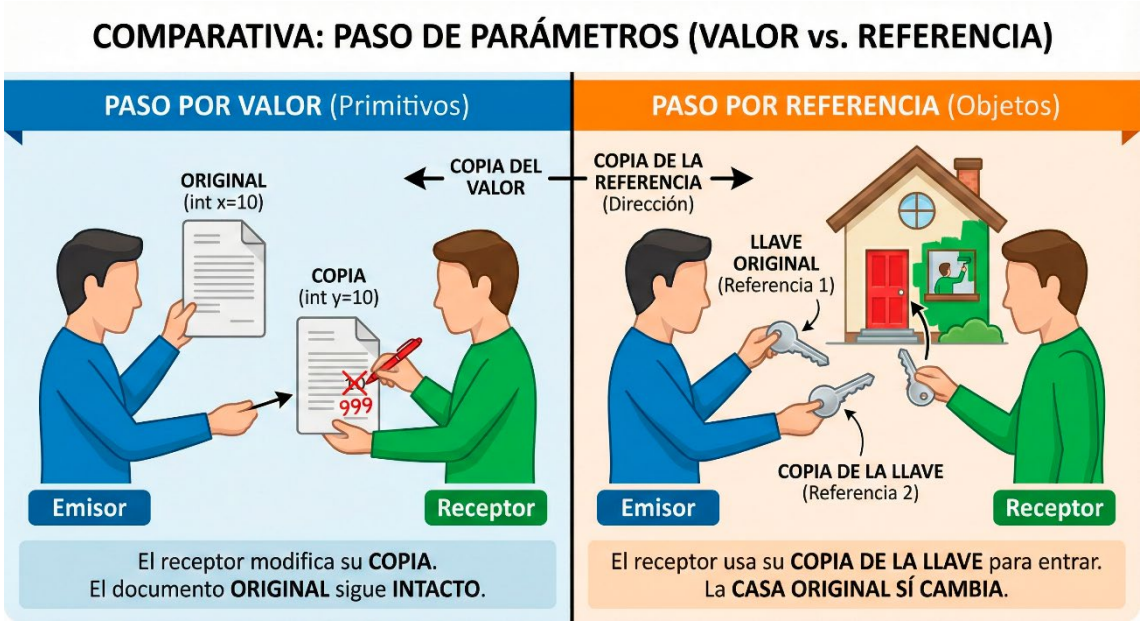
Recuerda que la variable de un objeto es un "Mando a Distancia".

Si pasas el objeto como parámetro, estás entregando una copia del mando a distancia. Ahora, tanto el código de fuera como el método de dentro tienen mandos que apuntan a la **MISMA TELEVISIÓN (Objeto en el Heap).**

- **Analogía:** Tienes la llave de tu casa. Haces una copia de la llave y se la das a un pintor (el método). El pintor entra en tu casa y pinta las paredes de verde. Cuando tú vuelves a casa con tu llave original, **las paredes están verdes**.

Código de demostración:
Java
```
public class Coche {
    String color = "Rojo";
}

public class PruebaObjetos {

    public void pintarCoche(Coche c) {
        // 'c' es una copia del mando, pero apunta al MISMO coche
        c.color = "Verde"; // Modificamos el objeto REAL en el Heap
    }

    public static void main(String[] args) {
        PruebaObjetos p = new PruebaObjetos();

        Coche miFerrari = new Coche();
        System.out.println("Color original: " + miFerrari.color);
// Rojo

        p.pintarCoche(miFerrari); // Pasamos la referencia

        // ¡El objeto original HA CAMBIADO!
        System.out.println("Color final: " + miFerrari.color); //
Verde
    }
}
```

Conclusión: Los objetos se pasan por referencia. Si el método modifica los atributos internos del objeto, **esos cambios son permanentes y afectan a tu variable original**.



C. El caso trampa: La Reasignación

¿Qué pasa si dentro del método, en vez de modificar un atributo (`.color`), intentamos cambiar el objeto entero (`= new Coche()`)?

Aquí volvemos a la analogía del mando. Si al pintor le das una copia de tu llave, y el pintor tira esa copia a la basura y se compra una llave de **otra** casa... tu casa original no cambia.

Java
```
public void cambiarCocheEntero(Coche c) {
    // Esto solo rompe el vínculo de la variable local 'c'.
    // NO afecta a 'miFerrari' de fuera.
    c = new Coche();
    c.color = "Azul";
}
// Al salir del método, 'miFerrari' sigue siendo Rojo y el coche
Azul se pierde.
```

Tabla Resumen

Tipo de Dato	¿Qué se envía?	¿Qué pasa si el método cambia el valor?	Efecto secundario
Primitivo (`int`, etc.)	Una **Copia del Valor**.	El original NO cambia.	Ninguno. Seguro.
Objeto (`Coche`, etc.)	Una **Copia de la Referencia** (Dirección).	El objeto original **SÍ cambia** (sus atributos).	Alto. El objeto vuelve modificado.

2.7 Reutilización de código

A. Herencia: Relación "Es Un" (Is-A)

La herencia permite crear una clase nueva (Hija / Subclase) basándose en una clase que ya existe (Padre / Superclase).

La clase hija hereda automáticamente todos los atributos y métodos visibles (`public` o `protected`) del padre. No tienes que volver a escribirlos.

- **Palabra clave:** `extends`.
- **Pregunta de control:** ¿El objeto B **ES UN** tipo de objeto A?
    - ¿Un Coche **ES UN** Vehículo? SÍ. (Herencia correcta).
    - ¿Una Rueda **ES UN** Coche? NO. (Herencia incorrecta).

Ejemplo: Imagina que ya existe una clase `Vehiculo` con código para moverse.

Java
```java
// CLASE PADRE (Superclase)
public class Vehiculo {
    public String marca;
    public void arrancar() {
        System.out.println("Brum brum...");
    }
}

// CLASE HIJA (Subclase)
// Al poner 'extends Vehiculo', Coche recibe todo lo de arriba
GRATIS.
public class Coche extends Vehiculo {
    int numeroPuertas; // Solo añadimos lo específico de un coche
}

// USO:
Coche miCoche = new Coche();
miCoche.marca = "Toyota"; // ¡Funciona! Lo heredó de Vehiculo.
miCoche.arrancar();       // ¡Funciona! Lo heredó de Vehiculo.
```

Ventaja: Si mañana mejoras el método `arrancar()` en la clase `Vehiculo`, todos los `Coches`, `Motos` y `Camiones` reciben la mejora automáticamente.

B. Composición: Relación "Tiene Un" (Has-A)

La composición (o agregación) consiste en construir objetos complejos utilizando objetos más simples como piezas de Lego. En lugar de heredar, **usas una clase como atributo de otra**.

Aquí no hay padres ni hijos, hay un objeto que **contiene** a otros.

- **Implementación:** Declaras una variable de instancia cuyo tipo es otra clase.
- **Pregunta de control:** ¿El objeto A **TIENE UN** objeto B dentro?
    - ¿Un Coche **TIENE UN** Motor? SÍ. (Composición correcta).
    - ¿Una Casa **TIENE UNA** Cocina? SÍ.

Ejemplo: Un Ordenador no "es un" procesador, pero "tiene un" procesador.

Java
```java
// Clase componente 1
public class Procesador {
    String modelo = "Intel i7";
}

// Clase componente 2
public class Pantalla {
    int pulgadas = 24;
}

// CLASE COMPUESTA (El contenedor)
public class Ordenador {
    // ATRIBUTOS DE TIPO OBJETO (Composición)
    private Procesador cpu;
    private Pantalla monitor;
```

```
    // En el constructor, inicializamos las piezas
    public Ordenador() {
        this.cpu = new Procesador();
        this.monitor = new Pantalla();
    }

    void encender() {
        System.out.println("Iniciando " + cpu.modelo + " en monitor
de " + monitor.pulgadas + "'");
    }
}
```

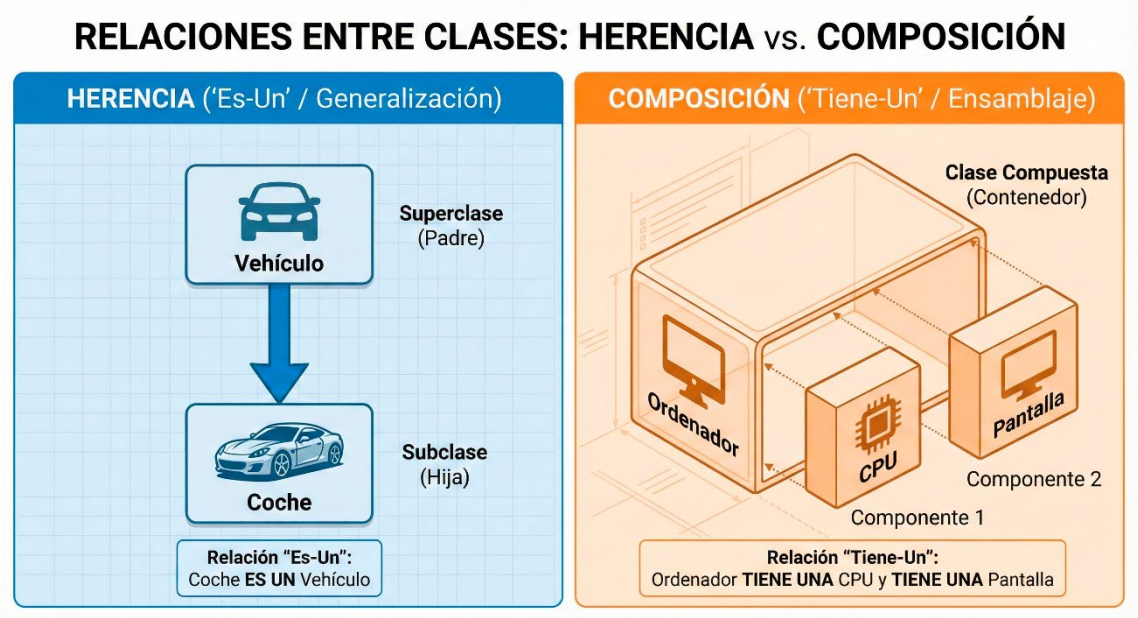
Ventaja: Permite crear sistemas modulares. Si quieres cambiar el procesador, solo cambias el objeto `cpu`, no tienes que tocar toda la clase `Ordenador`.

C. Tabla Comparativa: ¿Cuál elijo?

Un error de principiante es intentar usar Herencia para todo.

Estrategia	Relación	Ejemplo	¿Cuándo usarla?
Herencia (`extends`)	**Es-Un** (Generalización)	`Perro` **es un** `Animal`.	Cuando la nueva clase es una **versión más específica** de la original.
Composición (Atributo)	**Tiene-Un** (Ensamblaje)	`Perro` **tiene un** `Collar`.	Cuando la nueva clase **necesita usar** la funcionalidad de otra, pero no es del mismo tipo.

El Principio de Diseño: "Favorece la Composición sobre la Herencia". La composición es más flexible. La herencia es muy rígida (una vez que heredas, estás "casado" con la clase padre para siempre).



3. Organización y Documentación

3.1 Paquetes (`packages`): Organización lógica y control de acceso

A medida que un programa crece, pasamos de tener 2 o 3 archivos a tener cientos. Si dejáramos todos esos archivos sueltos en una sola carpeta, ocurrirían dos desastres: no encontraríamos nada y empezaríamos a tener conflictos de nombres (¿cuántas veces querrás llamar a una clase `Utils` o `Config`?).

Un **paquete** en Java es un contenedor que agrupa clases, interfaces y subpaquetes relacionados. Cumple una doble función vital: **Orden** y **Seguridad**.

A. Organización Lógica (Namespaces)

Físicamente, un paquete es un **directorio** (carpeta) en tu sistema operativo. Lógicamente, es un **espacio de nombres** (namespace) que garantiza que tus clases sean únicas.

1. Evitar colisiones de nombres: Java permite tener dos clases con el mismo nombre, siempre que estén en paquetes distintos.

- `java.util.Date` (Para fechas genéricas).
- java.sql.Date (Para fechas de bases de datos).

Java sabe cuál usar gracias al apellido (el paquete).

2. Convención de Nombres (El estándar mundial): Para asegurar que tu paquete sea único en todo el mundo, se utiliza el dominio de internet de tu empresa invertido. Se escribe siempre en minúsculas.

- Si tu web es `miempresa.com` y haces una app de facturación: `com.miempresa.facturacion`

3. Mapeo a Carpetas: Los puntos en el nombre del paquete representan niveles de carpetas.

- Paquete: `com.miempresa.juego.graficos`
- Ruta real: `.../src/com/miempresa/juego/graficos/`

B. Control de Acceso (La Muralla del Paquete)

Los paquetes no son solo cajones para ordenar; son fronteras de seguridad. Java utiliza los paquetes para definir quién puede ver qué. Aquí es donde cobran sentido los modificadores de acceso que vimos antes:

1. Acceso por Defecto (Package-Private): Si a una clase, atributo o método NO le pones ningún modificador (ni public, ni private, ni protected), Java le asigna visibilidad de paquete.

- **Regla:** "Solo mis vecinos de carpeta pueden verme".
- **Uso:** Clases auxiliares que solo sirven para ayudar a otras clases del mismo paquete, pero que no quieres exponer al resto de la aplicación.

2. Acceso Protegido (protected): Permite el acceso a las clases del mismo paquete (igual que el default) Y TAMBIÉN a las subclases que estén fuera del paquete.

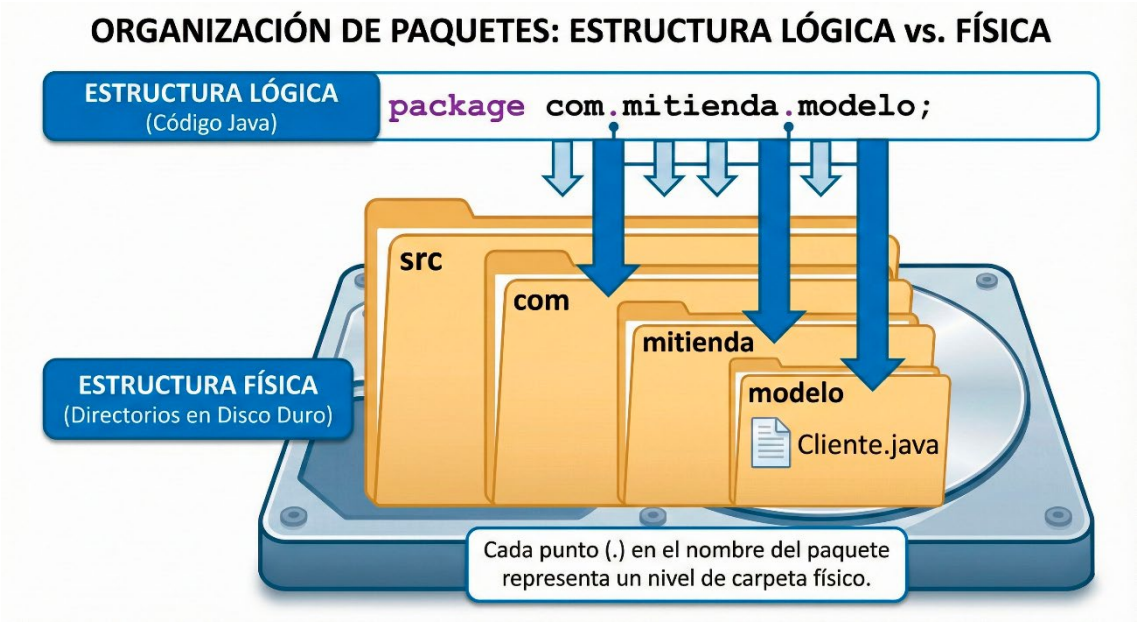
3. Acceso Público (public): Es la única forma de "sacar" algo del paquete. Si una clase no es public, es invisible para cualquier código que esté fuera de su carpeta.

C. Ejemplo de Arquitectura Real

Imagina una aplicación Tienda Online. No mezclamos todo. Creamos paquetes por funcionalidad:

Plaintext

```
src/
 └── com/
      └── mitienda/
           ├── main/
           │    └── Main.java        (Punto de entrada)
           ├── modelo/               (Datos puros)
           │    ├── Cliente.java
           │    └── Producto.java
           ├── base_datos/           (Lógica técnica)
           │    └── ConectorDB.java
           └── util/                 (Herramientas)
                └── CalculadoraImpuestos.java
```



En el código: Para indicar que un archivo pertenece a uno de estos cajones, la primera línea ejecutable del archivo .java debe ser la declaración del paquete.

Java
```
// Archivo: src/com/mitienda/modelo/Cliente.java

package com.mitienda.modelo; // declaramos "Soy de este paquete"

public class Cliente {
    // ...
}
```

3.2 Librerías: Importación, creación y uso

En el ecosistema Java, la unidad de distribución de software por excelencia es el archivo **JAR** y el mecanismo para conectarlos es el **import**.

A. Profundizando en la Importación (`import`)

Ya vimos que `import` sirve para "ver" clases de otros paquetes. Pero hay matices importantes que distinguen a un programador novato de uno experto.

1. El conflicto de nombres. ¿Qué pasa si quieres usar la clase List (Lista) de Java y también una clase List (Lista visual) de una librería de gráficos? Ambas se llaman igual.

Si haces esto, Java se lía:

Java
```
import java.util.List;
import java.awt.List; // ❌ ERROR: The import collides with another import
```

Solución: Importas una y usas la **Ruta Completamente Calificada** (Fully Qualified Name) para la otra.

Java
```
import java.util.List; // Importamos la que más usamos

public class MiClase {
    List listaNombres; // Usa java.util.List

    // Para la otra, escribimos la ruta completa
    java.awt.List listaVisual = new java.awt.List();
}
```

2. Importación Estática (import static). Sirve para importar métodos estáticos y constantes de una clase, de forma que puedas usarlos como si fueran tuyos, sin poner el nombre de la clase delante.

Java
```
// Importamos todo lo estático de Math
import static java.lang.Math.*;

public class Calculos {
    void calcular() {
        // En lugar de Math.sqrt(25) o Math.PI
        double x = sqrt(25);
        double y = PI * 2;
    }
}
```

B. Archivos .jar: El contenedor universal

Un archivo **JAR** (Java ARchive) es el formato estándar para distribuir aplicaciones y librerías en Java.

- **¿Qué es realmente?** No es magia, es literalmente un archivo **ZIP** con la extensión cambiada a `.jar`.
- **¿Qué contiene?**
    1. Las clases compiladas (`.class`).
    2. Recursos (imágenes, archivos de configuración).
    3. Un archivo especial llamado **MANIFEST.MF**.

Tipos de JAR:

1. **JAR de Librería:** Solo contiene código para ser usado por otros programas. No se puede ejecutar por sí solo (no tiene `main`).
2. **JAR Ejecutable:** Contiene un programa completo. Puedes hacer doble clic sobre él (o ejecutar `java -jar archivo.jar`) y arranca.
    - *El secreto:* Su archivo `MANIFEST.MF` tiene una línea extra: `Main-Class: com.miempresa.Inicio`, que le dice a Java dónde empezar.

Creación de archivos JAR: Aunque se puede hacer por consola (jar cf archivo.jar ...), hoy en día se hace desde el IDE (Eclipse, IntelliJ, NetBeans):

1. Vas a `File` > `Export` (o `Build Artifact`).
2. Eliges "Java JAR file".
3. Decides si quieres incluir el código fuente (`.java`) o solo el compilado (para proteger tu propiedad intelectual).

C. Uso de Librerías Externas

Java tiene una librería estándar enorme, pero no lo tiene todo. Si quieres crear PDFs, leer Excel, conectar con Twitter o crear juegos 3D, necesitarás librerías de terceros (como *Apache POI*, *iText* o *LibGDX*).

El concepto de CLASSPATH (La ruta de construcción). Java no busca en todo tu disco duro. Solo busca clases en las carpetas que tú le digas explícitamente. Esa lista de carpetas y JARs permitidos se llama Classpath.

Pasos para usar una librería externa (Método Manual):

1. **Descargar:** Bajas el archivo `.jar` de la web del autor (ej: `commons-lang3.jar`).
2. **Copiar:** Lo pegas en una carpeta de tu proyecto (normalmente llamada `lib` o `libs`).
3. **Configurar el Build Path:**
   - Haces clic derecho en el JAR dentro de tu IDE.
   - Seleccionas **"Add to Build Path"** o **"Add as Library"**.
4. **Programar:** Ahora ya puedes hacer `import org.apache.commons...` en tu código.

Nota Pro: Maven y Gradle En el mundo profesional moderno, casi nadie descarga JARs manualmente (es un infierno mantener versiones). Usamos **Gestores de Dependencias** como **Maven** o **Gradle**. Tú escribes en un archivo de texto: "Quiero la librería PDFBox versión 2.0", y Maven la descarga de internet, la añade al Classpath y la actualiza por ti automáticamente.

3.3 Documentación: Generación de documentación técnica (Javadoc)

Cuando compras una lavadora, no necesitas abrir el motor para saber cómo funciona; simplemente lees el manual de usuario.

Cuando escribes una clase para que la usen otros programadores (o tú mismo en el futuro), no deberían tener que leer tu código línea por línea para saber cómo usarla. Deberían tener un "manual".

Javadoc es el estándar de la industria para generar documentación de la API (Interfaz de Programación de Aplicaciones) directamente desde el código fuente.

A. Diferencia entre Comentarios y Javadoc

- **Comentarios normales (`//` o `/* ... */`):**
    - **Destinatario:** El programador que está **modificando** el código por dentro.
    - **Contenido:** "Aquí sumo 1 porque el array empieza en 0", "Algoritmo temporal".
    - **Resultado:** El compilador los ignora y desaparecen.

- **Comentarios Javadoc (`/** ... */`):**
    - **Destinatario:** El programador que va a **consumir/usar** tu clase (caja negra).
    - **Contenido:** Qué hace el método, qué parámetros necesita y qué devuelve.
    - **Resultado:** Se procesan para crear una página web (HTML) de documentación.

B. Sintaxis y Etiquetas (Tags)

Los comentarios Javadoc deben colocarse **justo encima** de la declaración de la clase, atributo o método que describen. Empiezan con `/**` y terminan con `*/`.

Dentro, utilizamos **etiquetas** (que empiezan por `@`) para estructurar la información:

Etiqueta	Uso	Dónde se pone
`@author`	Nombre del creador de la clase.	Encima de la Clase
`@version`	Versión actual del código.	Encima de la Clase
`@param`	Describe qué es y qué formato debe tener un parámetro.	Encima de Métodos
`@return`	Explica qué dato devuelve el método.	Encima de Métodos
`@throws`	Indica qué errores (Excepciones) puede provocar.	Encima de Métodos
`@see`	Crea un enlace a otra parte de la documentación ("Ver también").	En cualquier sitio

C. Ejemplo Práctico

Veamos cómo documentar una clase de utilidades matemáticas. Fíjate cómo explicamos el contrato (qué entra y qué sale) sin explicar la matemática interna.

Java
```
/**
 * Clase de utilidades para realizar cálculos geométricos.
 * Permite calcular áreas y perímetros de figuras estándar.
 * * @author Gemini AI
 * @version 1.0
 */
```

```java
public class Geometria {

    /**
     * Valor aproximado del número Pi para cálculos de precisión
media.
     */
    public static final double PI = 3.14159;

    /**
     * Calcula el área de un círculo dado su radio.
     * * @param radio La distancia desde el centro hasta el borde.
     * Debe ser un número positivo.
     * @return El área calculada en las mismas unidades que el
radio al cuadrado.
     * Devuelve -1 si el radio es negativo.
     */
    public double calcularAreaCirculo(double radio) {
        if (radio < 0) return -1;
        return PI * radio * radio;
    }
}
```

D. La Generación del HTML

Escribir los comentarios es solo la mitad del trabajo. La magia ocurre cuando **generamos** la documentación.

1. **Herramienta `javadoc`:** El JDK (Java Development Kit) trae una herramienta de línea de comandos llamada javadoc. Si ejecutas javadoc MiClase.java, te creará varios archivos .html.
2. **Desde el IDE (La forma habitual):** En entornos como Eclipse, IntelliJ o NetBeans, es automático:

   - **Eclipse:** Menú `Project` > `Generate Javadoc...`
   - **IntelliJ:** Menú `Tools` > `Generate JavaDoc...`

El resultado es una web idéntica a la documentación oficial de Java que consultas en internet (porque la de Oracle también se hace así).

Ventaja en el IDE: Una vez que documentas tu código con Javadoc, cuando otro programador use tu clase y ponga el ratón encima de tu método (Geometria.calcularAreaCirculo), el IDE mostrará una ventana flotante con tu explicación formateada. ¡Parecerá una clase oficial de Java!

Resumen

1. Uso de Objetos (El Rol de Consumidor)

Nos centramos en cómo gestionar objetos en la memoria RAM.

- **Ciclo de Vida:**

  1. **Declaración:** `Coche c;` (Crea la referencia/mando en el Stack. Valor `null`).

2. **Instanciación:** `new Coche` (Reserva memoria física en el Heap).
3. **Inicialización:** `...Coche()` (Llama al constructor para preparar los datos).

- **Gestión de Memoria:**
  - **Stack:** Memoria rápida y temporal. Guarda variables locales y **referencias**.
  - **Heap:** El gran almacén. Guarda los **objetos** reales.
  - **Garbage Collector:** El basurero automático que borra los objetos del Heap cuando se quedan sin referencias (`c = null`).

- **Miembros Estáticos (`static`):** Pertenecen a la **Clase**, no al objeto. Son compartidos por todos (ej: `Math.PI`).

2. Desarrollo de Clases (El Rol de Arquitecto)

Aprendimos a escribir las plantillas (`.java`) que definen cómo son los objetos.

Estructura Interna

- **Atributos (Estado):** Variables dentro de la clase. Definen "qué sabe" el objeto.
  - *Shadowing:* Si una variable local se llama igual que un atributo, usamos **`this.atributo`** para diferenciarlos.
- **Métodos (Comportamiento):** Funciones dentro de la clase. Definen "qué hace" el objeto.
  - *Sobrecarga:* Mismo nombre de método, distintos parámetros.

Constructores

- Método especial que se ejecuta al hacer `new`.
- **Regla:** Si no escribes ninguno, Java te regala uno vacío (por defecto). Si escribes uno con parámetros, el vacío desaparece (a menos que lo escribas tú también).

Encapsulamiento (Seguridad)

Es la práctica de proteger los datos sensibles.

- **Niveles de Acceso:**
  1. `private`: Solo la propia clase (La caja fuerte).
  2. *(default)*: Solo el paquete (Los vecinos).
  3. `protected`: Paquete + Hijos (Herencia).
  4. `public`: Todo el mundo.
- **Patrón Oro:** Atributos `private`, Métodos Getters/Setters `public`.

Paso de Parámetros

- **Primitivos:** Se pasa una **copia del valor**. El original está a salvo.
- **Objetos:** Se pasa una **copia de la referencia** (el mando). Si el método modifica el objeto, **el original cambia**.

Reutilización

- **Herencia (`extends`):** Relación "Es-Un". `Coche` es un `Vehículo`. Hereda código.
- **Composición:** Relación "Tiene-Un". `Coche` tiene un `Motor`. Usa objetos dentro de objetos.

3. Organización y Documentación (El Rol de Bibliotecario)

Cómo mantener el código ordenado para proyectos grandes.

- **Paquetes (`package`):** Carpetas lógicas para agrupar clases y evitar choques de nombres (`com.miempresa.modelo`).
- **Imports:** Necesarios para usar clases que viven en otros paquetes.
- **Librerías (`.jar`):** Archivos ZIP con clases compiladas para compartir o usar código de terceros.
- **Javadoc (`/** ... */`):** Comentarios especiales que generan documentación HTML automática para explicar la API de tu clase (`@param`, `@return`).

Ejemplo: La Clase Perfecta

Java
```java
package com.empresa.proyecto; // 1. Organización

/**
 * Clase que representa un usuario del sistema.
 * @author Gemini
 */
public class Usuario { // 2. Definición de Clase

    // 3. Atributos PRIVADOS (Encapsulamiento)
    private String nombre;
    private int edad;
    public static int contadorUsuarios = 0; // Estático
(Compartido)

    // 4. Constructor (Inicialización)
    public Usuario(String nombre, int edad) {
        this.nombre = nombre; // Uso de 'this' para evitar
confusión
        this.edad = edad;
        contadorUsuarios++;
    }

    // 5. Métodos PÚBLICOS (Interfaz)
    public void saludar() {
        System.out.println("Hola, soy " + this.nombre);
    }

    // Getters y Setters
    public String getNombre() { return nombre; }
```

}